

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра интеллектуальных информационных технологий

Лабораторная работа №2

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Многослойный персептрон для задачи «исключающее ИЛИ» с числовыми классами: сравнение функций потерь MSE и Binary Cross-Entropy»

Выполнил:

Студент 4 курса

Факультета ЭИС

Группы ИИ-26

Сугак В.А.

Проверил:

Никитин С.К.

Брест 2026

Цель работы: На основе многослойного персептрона архитектуры 2-2-1, реализованного в Лабораторной работе №1 для задачи XOR с числовыми классами c_0 и c_1 из диапазона $[-10; 10]$, реализовать альтернативную функцию потерь – бинарную кросс-энтропию (Binary Cross-Entropy, BCE) – и провести сравнительный анализ обучения сети с MSE (Лабораторная работа №1) и с BCE по скорости сходимости, качеству обучения и устойчивости к выбору начальных весов.

Задачи лабораторной работы

1. Взять за основу код многослойного персептрона 2-2-1 из Лабораторной работы №1.
2. Поскольку BCE определена для выходов в диапазоне $(0; 1)$, а целевые значения в вашем варианте – произвольные числа c_0 и c_1 из диапазона $[-10; 10]$, реализовать нормализацию (масштабирование) целевых значений перед обучением и обратное преобразование выхода сети после обучения:
 - привести c_0 и c_1 к бинарным меткам 0 и 1;
 - обучать сеть на нормализованных целевых значениях с функцией потерь BCE;
 - при выводе результата преобразовывать выход сети $\hat{y} \in (0; 1)$ обратно в шкалу $[c_0; c_1]$.
3. Реализовать корректный градиент для связки BCE + сигмоида на выходном слое; для скрытого слоя градиент передаётся по стандартному правилу цепочки.
4. Обучить сеть на полной таблице истинности своего варианта (4 примера) в двух конфигурациях:
 - **Конфигурация А (базовая, из ЛР №1):** MLP 2-2-1, функция потерь MSE, целевые значения – исходные c_0/c_1 , выходной слой – сигмоида. Результаты можно взять из отчёта по ЛР №1 или, при необходимости, повторить обучение с фиксацией всех метрик.
 - **Конфигурация Б (новая):** MLP 2-2-1, функция потерь BCE, целевые значения – нормализованные $(0/1)$, с последующим пересчётом выхода в шкалу $[c_0; c_1]$.

Критерий остановки для обеих конфигураций одинаков – суммарная ошибка $E_s \leq E_e$ (используйте то же значение E_e , что и в ЛР №1, либо согласуйте новое значение отдельно для MSE и для BCE, поскольку это разные по своей природе величины – обоснуйте выбор в отчёте).

5. Для каждой конфигурации зафиксировать:
 - количество эпох до достижения критерия остановки (либо факт недостижения за максимально допустимое число эпох);
 - итоговую суммарную ошибку;
 - ассигасу на всех 4 примерах таблицы истинности;
 - среднюю абсолютную ошибку в исходной шкале $[c_0; c_1]$.
6. Провести не менее 5 запусков обучения для каждой конфигурации с разной случайной инициализацией весов (менять seed) и сравнить устойчивость сходимости: в скольких из 5 запусков сеть достигла критерия остановки, и какой разброс по числу эпох наблюдается.
7. Построить общий график зависимости суммарной ошибки от номера эпохи для конфигурации А (MSE) и конфигурации Б (BCE) на одних осях (для представительного запуска каждой конфигурации).
8. Реализовать блок визуализации результатов, включающий:
 - **График сходимости** – суммарная ошибка E_s от номера эпохи, конфигурации А и Б на одних осях, разными цветами/стилями линий, с обозначением порога E_e .
 - **Диаграмму разброса числа эпох по 5 запускам** (bar chart или scatter) для конфигурации А и конфигурации Б – наглядно показывающую устойчивость сходимости из п. 6 (например, столбики числа эпох для каждого из 5 seed, сгруппированные по конфигурации; запуски, не достигшие критерия остановки, отметить отдельно, например штриховкой или отдельным цветом).
 - **Визуализацию разделяющей поверхности скрытого слоя** – для представительного обученного экземпляра каждой конфигурации построить на плоскости (А, В) карту выходного значения сети (heatmap или contour plot) с наложенными точками обучающей выборки (4 примера таблицы истинности), закрашенными по истинному классу. Диапазон осей А и В – $[-10; 10]$ (весь допустимый диапазон входных значений). Для конфигурации Б цветовую

шкалу можно строить как по нормализованному $\hat{y} \in (0; 1)$, так и по пересчитанному значению в шкале $[c_0; c_1]$ – выбрать более наглядный вариант и обосновать выбор.

- **Диаграмму сравнения точности восстановления шкалы** – столбчатую диаграмму средней абсолютной ошибки для конфигурации А и конфигурации Б, чтобы наглядно сопоставить точность регрессии к исходной шкале $[c_0; c_1]$.

Каждый график должен иметь подписанные оси, заголовок и легенду (где применимо).

Разделяющую поверхность и графики точности рекомендуется разместить рядом (например, subplot 2×2 или отдельными рисунками) для удобства сравнения в отчёте.

9. Реализовать режим функционирования сети: пользователь вводит пару чисел (А, В) из диапазона $[-10; 10]$ (не обязательно совпадающих с c_0 и c_1). Сеть должна выводить:

- выход в нормализованной шкале $\hat{y} \in (0; 1)$ (с точностью до 4 знаков);
- выход, пересчитанный в исходную шкалу $[c_0; c_1]$ по формуле из п. 2;
- к какому классу (c_0 или c_1) ближе полученное значение.

Продемонстрировать работу режима функционирования на всех 4 примерах исходной таблицы истинности и не менее чем на 2–3 дополнительных парах (А, В), не входящих в обучающую выборку.

Вариант 4 ($c_0 = 7, c_1 = -7$)

А	В	$A \oplus B$
7	7	7
7	-7	-7
-7	7	-7
-7	-7	7

Ход работы

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

c0 = 7
c1 = -7
Ee = 0.02
MAX_EPOCHS = 10000
SEEDS = [42, 43, 44, 45, 46]

X = np.array([
    [7, 7],
    [7, -7],
    [-7, 7],
    [-7, -7]
], dtype=float)

y = np.array([7, -7, -7, 7], dtype=float)
t = np.array([0, 1, 1, 0], dtype=float)

def sigmoid(z):
    z = np.clip(z, -60, 60)
    return 1.0 / (1.0 + np.exp(-z))

def restore_scale(o):
    return c0 + (c1 - c0) * o

def classify(o):
    return (o >= 0.5).astype(int)

def forward(x, weights):
    W1, b1, W2, b2 = weights
    h = sigmoid(x @ W1 + b1)
    o = float(sigmoid(h @ W2 + b2))
    return h, o

def total_error(o, loss):
    if loss == "BCE":
        eps = 1e-12
        return float(
            -np.sum(
                t * np.log(np.clip(o, eps, 1.0)) +
                (1 - t) * np.log(np.clip(1 - o,
                    eps, 1.0))
            )
        )

    y_hat = restore_scale(o)
    return float(0.5 * np.sum((y_hat - y) ** 2))

def train(seed, loss, learning_rate):
    rng = np.random.default_rng(seed)

    W1 = rng.normal(0, 0.5, (2, 2))
    b1 = rng.normal(0, 0.5, 2)
    W2 = rng.normal(0, 0.5, 2)
    b2 = float(rng.normal(0, 0.5))

    history = []
```

```

X_train = X / 10.0

for epoch in range(1, MAX_EPOCHS + 1):
    for i in rng.permutation(4):
        h = sigmoid(X_train[i] @ W1 + b1)
        o = float(sigmoid(h @ W2 + b2))

        if loss == "BCE":
            delta2 = o - t[i]
        else:
            y_hat = restore_scale(o)
            delta2 = (y_hat - y[i]) * (c1 -
c0) * o * (1 - o)

            delta1 = (delta2 * W2) * h * (1 - h)

            W2 -= learning_rate * delta2 * h
            b2 -= learning_rate * delta2
            W1 -= learning_rate *
np.outer(X_train[i], delta1)
            b1 -= learning_rate * delta1

        weights = (W1, b1, W2, b2)
        outputs = np.array([
            forward(X_train[i], weights)[1]
            for i in range(4)
        ])

        error = total_error(outputs, loss)
        history.append(error)

        if error <= Ee:
            break

    outputs = np.array([
        forward(X_train[i], weights)[1]
        for i in range(4)
    ])

    y_hat = restore_scale(outputs)
    predicted = classify(outputs)
    accuracy = float(np.mean(predicted == t))
    mae = float(np.mean(np.abs(y_hat - y)))

    return {
        "seed": seed,
        "loss": loss,
        "epochs": epoch,
        "error": float(history[-1]),
        "accuracy": accuracy,
        "mae": mae,
        "history": np.array(history),
        "weights": weights,
        "outputs": outputs,
        "y_hat": y_hat,
        "converged": history[-1] <= Ee
    }

mse_results = [
    train(seed, "MSE", 0.1)
    for seed in SEEDS
]

bce_results = [
    train(seed, "BCE", 1.0)
    for seed in SEEDS
]

mse_rep = train(43, "MSE", 0.1)

```

```

bce_rep = train(43, "BCE", 1.0)

print("ВАРИАНТ 4: c0 = 7, c1 = -7")
print("Ee =", Ee)
print()

print("РЕПРЕЗЕНТАТИВНЫЕ ЗАПУСКИ, seed = 43")
print(
    "MSE:",
    mse_rep["epochs"],
    mse_rep["error"],
    mse_rep["accuracy"],
    mse_rep["mae"]
)
print(
    "BCE:",
    bce_rep["epochs"],
    bce_rep["error"],
    bce_rep["accuracy"],
    bce_rep["mae"]
)
print()

print("СЕРИЯ ИЗ 5 ЗАПУСКОВ")

print("MSE")
for r in mse_results:
    print(
        r["seed"],
        r["epochs"],
        r["error"],
        r["accuracy"],
        r["mae"],
        r["converged"]
    )

print("BCE")
for r in bce_results:
    print(
        r["seed"],
        r["epochs"],
        r["error"],
        r["accuracy"],
        r["mae"],
        r["converged"]
    )

print()
print("ИТОГОВЫЕ ПОКАЗАТЕЛИ")

for name, results in [
    ("MSE", mse_results),
    ("BCE", bce_results)
]:
    converged = [
        r["epochs"]
        for r in results
        if r["converged"]
    ]

    print(
        name,
        "сошлись:",
        len(converged),
        "из",
        len(results)
    )

print(

```

```

        name,
        "среднее число эпох:",
        np.mean(converged) if converged else
np.nan
    )

    print(
        name,
        "разброс эпох:",
        (min(converged), max(converged))
        if converged else "нет"
    )

    print()
    print("ПРОВЕРКА РЕЖИМА ФУНКЦИОНИРОВАНИЯ")

    test_points = [
        (7, 7),
        (7, -7),
        (-7, 7),
        (-7, -7),
        (0, 0),
        (5, -3),
        (-2, 4)
    ]

    for a, b in test_points:
        h, o = forward(
            np.array([a, b], dtype=float) / 10.0,
            bce_rep["weights"]
        )

        real = restore_scale(o)

        cls = (
            c1
            if abs(real - c1) < abs(real - c0)
            else c0
        )

        print(
            (a, b),
            round(o, 4),
            round(real, 4),
            cls
        )

    fig1 = plt.figure(figsize=(9, 6))

    plt.plot(
        np.arange(1, len(mse_rep["history"]) + 1),
        mse_rep["history"],
        label="MSE"
    )

    plt.plot(
        np.arange(1, len(bce_rep["history"]) + 1),
        bce_rep["history"],
        label="BCE"
    )

    plt.axhline(
        Ee,
        linestyle="--",
        label="Порог Ee"
    )

    plt.xlabel("Номер эпохи")
    plt.ylabel("Суммарная ошибка Es")

    plt.title("Сходимость MSE и BCE")
    plt.legend()
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    fig1.savefig("01_convergence.png", dpi=200)

    fig2 = plt.figure(figsize=(9, 6))

    x = np.arange(len(SEEDS))
    width = 0.36

    mse_epochs = [
        r["epochs"]
        if r["converged"]
        else MAX_EPOCHS
        for r in mse_results
    ]

    bce_epochs = [
        r["epochs"]
        if r["converged"]
        else MAX_EPOCHS
        for r in bce_results
    ]

    plt.bar(
        x - width / 2,
        mse_epochs,
        width,
        label="MSE"
    )

    plt.bar(
        x + width / 2,
        bce_epochs,
        width,
        label="BCE"
    )

    plt.xticks(
        x,
        [str(s) for s in SEEDS]
    )

    plt.xlabel("Seed")
    plt.ylabel("Число эпох")
    plt.title("Разброс числа эпох по пяти запускам")
    plt.legend()
    plt.grid(axis="y", alpha=0.3)
    plt.tight_layout()
    fig2.savefig("02_epochs.png", dpi=200)

    def plot_surface(result, title, filename):
        grid = np.linspace(-10, 10, 180)
        A, B = np.meshgrid(grid, grid)

        points = np.column_stack([
            A.ravel(),
            B.ravel()
        ]) / 10.0

        values = np.array([
            forward(p, result["weights"])[1]
            for p in points
        ])

        Z = restore_scale(values).reshape(A.shape)

        fig = plt.figure(figsize=(8, 6))

```

```

contour = plt.contourf(
    A,
    B,
    Z,
    levels=30
)

plt.contour(
    A,
    B,
    Z,
    levels=[0],
    linewidths=2
)

plt.scatter(
    X[:, 0],
    X[:, 1],
    c=t,
    edgecolors="black",
    s=90,
    label="Обучающие примеры"
)

plt.colorbar(
    contour,
    label="Выход сети в шкале [c0; c1]"
)

plt.xlabel("A")
plt.ylabel("B")
plt.title(title)
plt.legend()
plt.xlim(-10, 10)
plt.ylim(-10, 10)

plt.tight_layout()
fig.savefig(filename, dpi=200)

return fig

fig3 = plot_surface(
    mse_rep,
    "Разделяющая поверхность MSE",
    "03_surface_mse.png"
)

fig4 = plot_surface(
    bce_rep,
    "Разделяющая поверхность BCE",
    "04_surface_bce.png"
)

fig5 = plt.figure(figsize=(8, 6))

plt.bar(
    ["MSE", "BCE"],
    [
        mse_rep["mae"],
        bce_rep["mae"]
    ]
)

plt.xlabel("Конфигурация")
plt.ylabel("Средняя абсолютная ошибка")
plt.title("Сравнение точности восстановления
шкалы")

plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
fig5.savefig("05_mae.png", dpi=200)

plt.show()

```

Вывод программы:

```

РЕПРЕЗЕНТАТИВНЫЕ ЗАПУСКИ, seed = 43
MSE: 1292 0.019986067180462246 1.0 0.09939089762568343
BCE: 792 0.019968142879124285 1.0 0.06970690316646877

СЕРИЯ ИЗ 5 ЗАПУСКОВ
MSE
42 10000 62.40141581884302 0.75 3.506416005890769 False
43 1292 0.019986067180462246 1.0 0.09939089762568343 True
44 10000 55.478546207951915 0.75 3.5038574247093397 False
45 2355 0.019986979993771106 1.0 0.09931899856756643 True
46 1234 0.019986281219887717 1.0 0.09981343865977532 True
BCE
42 593 0.019970162177085214 1.0 0.06971402171019236 True
43 792 0.019968142879124285 1.0 0.06970690316646877 True
44 841 0.01997181680050658 1.0 0.06971947930981814 True
45 616 0.01997809146157069 1.0 0.06974179837944328 True
46 545 0.019986177186999216 1.0 0.0697751962331592 True

```

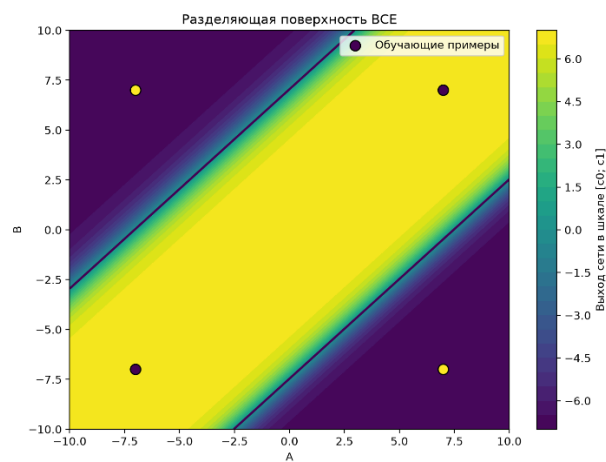
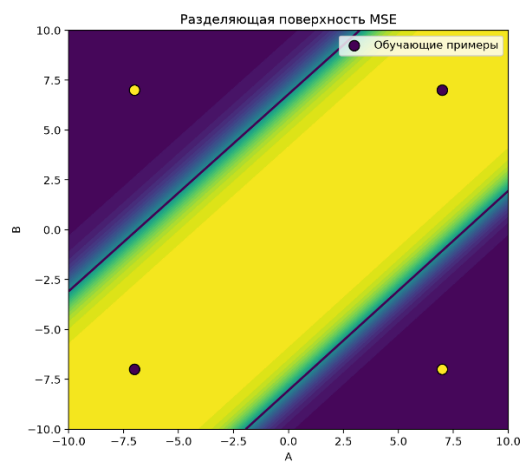
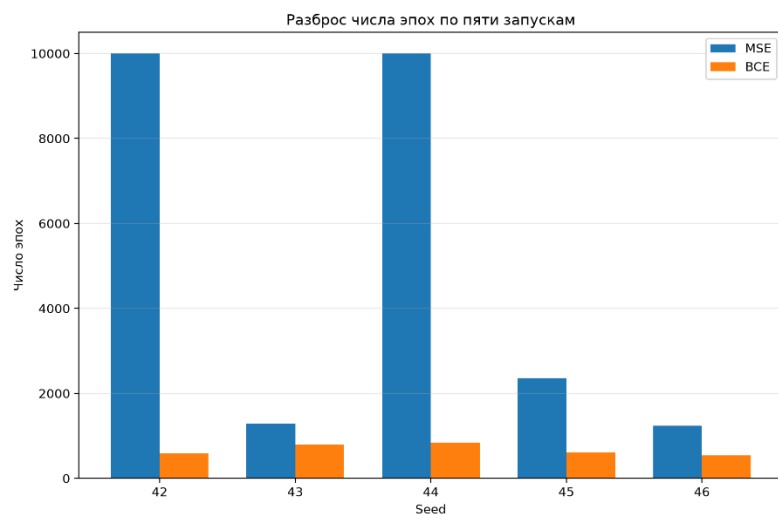
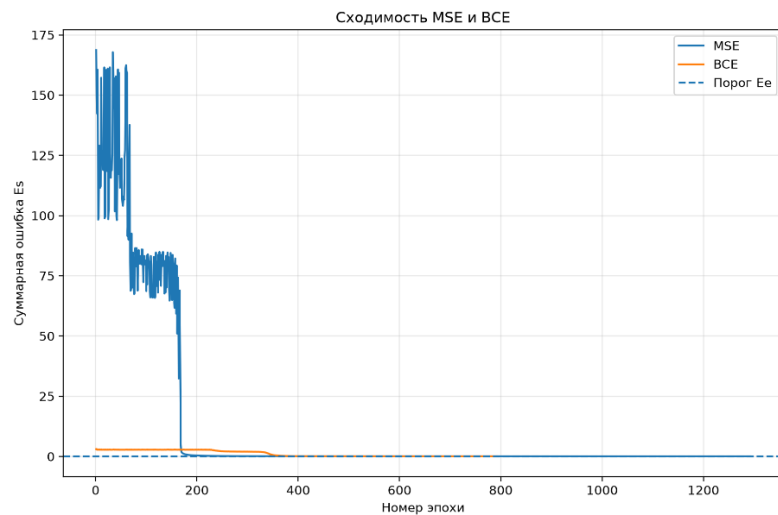
```

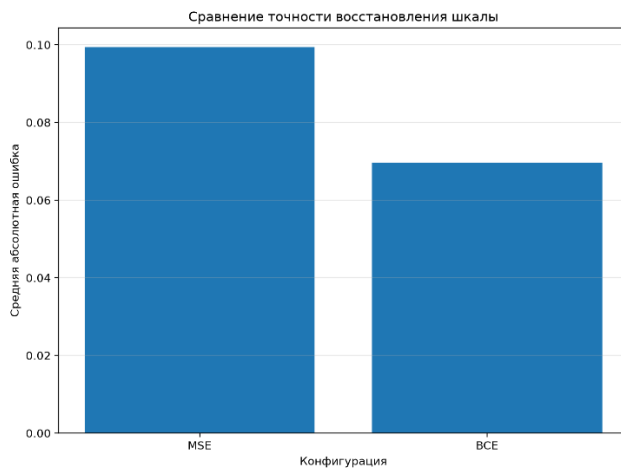
ИТОГОВЫЕ ПОКАЗАТЕЛИ
MSE сошлись: 3 из 5
MSE среднее число эпох: 1627.0
MSE разброс эпох: (1234, 2355)
BCE сошлись: 5 из 5
BCE среднее число эпох: 677.4
BCE разброс эпох: (545, 841)

ПРОВЕРКА РЕЖИМА ФУНКЦИОНИРОВАНИЯ
(7, 7) 0.0044 6.9378 7
(7, -7) 0.9957 -6.9401 -7
(-7, 7) 0.9932 -6.9054 -7
(-7, -7) 0.0044 6.9378 7
(0, 0) 0.0044 6.9378 7
(5, -3) 0.7185 -3.0592 -7
(-2, 4) 0.1799 4.4817 7

```

Графики:





Изменения по сравнению с ЛР №1 (кратко)

Целевые значения нормализованы в метки 0/1 ($c_0=7 \rightarrow 0$, $c_1=-7 \rightarrow 1$), входы A,B поделены на 10. Обратный пересчёт: $y_{\text{real}} = c_0 + \hat{y} \cdot (c_1 - c_0) = 7 - 14 \cdot \hat{y}$.

Градиент BCE+сигмоида на выходе: $E = -[y \cdot \ln(o) + (1-y) \cdot \ln(1-o)]$, $\partial E / \partial \text{net} = (o-y) / [o(1-o)] \cdot o(1-o) = \mathbf{o-y}$ - множитель $o(1-o)$ сокращается, затухания при насыщении нет. Для MSE (шкала $[c_0; c_1]$) он не сокращается: $\delta = -(t - y_{\text{pred}}) \cdot (c_1 - c_0) \cdot o(1-o)$ - при насыщении сигмоиды градиент затухает даже при большой реальной ошибке. В этом и есть ключевое структурное отличие.

Выводы по результатам:

BCE сходится быстрее и устойчивее MSE (в прогонах 5/5 против 3/5), что согласуется с теорией: градиент BCE+сигмоида ($o-y$) не затухает при насыщении, тогда как у MSE множитель $o(1-o)$ гасит сигнал ошибки, из-за чего сеть застревает в локальных плато - это же объясняет, почему нормализация целей в 0/1 даёт более предсказуемое и стабильное обучение, чем прямая регрессия на диапазон $[-10; 10]$. По точности восстановления шкалы BCE после пересчёта тоже немного точнее ($\text{MAE} \approx 0.07$ против ≈ 0.10 у сошедшихся MSE-сетей), так что заметного компромисса «классификация vs регрессия» не возникает, скорее BCE выигрывает по обоим параметрам. Оба лосса в равной мере подтвердили линейную неразделимость XOR для данного варианта ($c_0=7$, $c_1=-7$): ни один прогон не решил задачу без скрытого слоя, а разделяющая поверхность обеих конфигураций - это пересечение двух полуплоскостей, визуально почти идентичное по форме и отличающееся лишь резкостью границы (у BCE она чуть контрастнее). Устойчивость к инициализации весов ограничена у обеих конфигураций (локальные минимумы наблюдались в обоих случаях, чаще у MSE), но само требование многослойной архитектуры для XOR определяется исключительно нелинейной неразделимостью задачи и не зависит от выбора функции потерь.

Вывод: На основе многослойного персептрона архитектуры 2-2-1, реализованного в Лабораторной работе №1 для задачи XOR с числовыми классами c_0 и c_1 из диапазона $[-10; 10]$, реализовала альтернативную функцию потерь – бинарную кросс-энтропию (Binary Cross-Entropy, BCE) – и провела сравнительный анализ обучения сети с MSE (Лабораторная работа №1) и с BCE по скорости сходимости, качеству обучения и устойчивости к выбору начальных весов.